

Software-Projekt [SS25]

FhatGPT

Eingereicht von:

Laura Lopes

E-Mail: stud105703@fh-wedel.de

Sebastian Wagner

E-Mail: stud105316@fh-wedel.de

Christian Hinck

E-Mail: stud104315@fh-wedel.de

Hubertus Bungert

E-Mail: stud105639@fh-wedel.de

Inhaltsverzeichnis

Inhaltsverzeichnis.....	2
1. Benutzerhandbuch.....	3
1.1. Ablaufbedingungen.....	4
1.2. Programminstallation/Programmstart.....	4
1.3. Bedienungsanleitung.....	4
1.3.1. Zielsetzung des Chatbots.....	4
1.3.2. Grundlegende Regeln.....	5
1.3.3. Aufbau der Benutzeroberfläche.....	5
1.3.4. Nutzung des Chatbots.....	5
1.4. Fehlermeldungen.....	5
2. Programmiererhandbuch.....	7
2.1. Entwicklungskonfiguration.....	7
2.2. Problemanalyse und Realisation.....	8
2.2.1. Nutzung von Gemini mit Streamlit.....	8
2.2.2. Eigenes Frontend mit Gemini.....	8
2.2.3. Einführung einer optimierten Vektordatenbank.....	9
2.2.4. Nutzung von DeepSeek.....	9
2.3. Programmorganisationsplan.....	10
2.3.1 app.py.....	11
2.3.2 chatbot_logic.py.....	11
2.3.3 vector_store_manager.py.....	11
2.3.4 sync_scheduler.py.....	11
2.4. Dateien.....	11
2.5. Programmtests.....	12
2.6 Server.....	13

1. Benutzerhandbuch

Das Projekt FhatGPT wird im Rahmen eines Softwareprojektes von Laura Fernandes Lopes, Hubertus Bungert, Jonathan Herden, Christian Hinck und Sebastian Wagner umgesetzt. Jedes Teammitglied bringt unterschiedliche Stärken ein, sodass sowohl technische als auch organisatorische Aspekte in die Zusammenarbeit einfließen.

Ziel des Projekts ist die Entwicklung eines Chatbots für die Fachhochschule Wedel, der als digitaler Informationsassistent fungiert. Er soll Studieninteressierten und eingeschriebenen Studierenden gleichermaßen helfen, relevante Informationen schnell und unkompliziert zu erhalten. Studieninteressierte können sich dadurch ohne lange Suche in Modulhandbüchern über Inhalte, Module oder organisatorische Abläufe informieren. Studierende profitieren ebenfalls, indem der Chatbot Fragen zu Prüfungsordnung oder Studienorganisation beantwortet und das mühsame Durchsuchen umfangreicher Dokumente überflüssig macht. Langfristig soll FhatGPT in die digitale Infrastruktur der Hochschule integriert werden, um die Kommunikation zwischen Hochschule und Nutzenden zu verbessern.

Die Entscheidung für dieses Projekt beruht sowohl auf der fachlichen Herausforderung als auch auf dem klaren praktischen Nutzen. Die Entwicklung eines Chatbots, der auf umfangreichen und formalen Wissensquellen basiert, ist technisch anspruchsvoll. Sie erfordert ein tiefes Verständnis für Sprachmodelle, die Strukturierung von Daten sowie die Gestaltung einer benutzerfreundlichen Dialogführung. Gleichzeitig wird ein echter Mehrwert geschaffen, da wiederkehrende Fragen automatisiert beantwortet und Informationswege erheblich verkürzt werden. Für die Hochschule entsteht zudem ein Innovationspotenzial, weil mit einem digitalen Assistenten ein modernes Serviceangebot geschaffen wird.

Die Funktionsweise von FhatGPT ist einfach erklärt. Zu Beginn einer Interaktion begrüßt der Chatbot die Nutzerin oder den Nutzer und erfragt das Anliegen. Daraufhin kann eine konkrete Frage gestellt werden, etwa nach Modulen in einem bestimmten Semester oder nach vorgesehenen Prüfungsleistungen. Die Eingabe wird analysiert und mit der Wissensbasis abgeglichen, die aus den aktuellen Modulhandbüchern und der gültigen Prüfungsordnung besteht. Die Informationen werden herausgefiltert und in verständlicher Form zurückgegeben. Ein besonderer Mehrwert liegt darin, dass der Chatbot den Gesprächsverlauf berücksichtigt. Er erinnert sich an vorherige Fragen und bezieht diese in neue Antworten ein. Dadurch entsteht ein natürlicher Dialog mit konsistenten Ergebnissen.

Die Wissensbasis ist das zentrale Fundament von FhatGPT. Durch die Integration von Modulhandbüchern und Prüfungsordnungen ist sichergestellt, dass die Informationen aktuell und zuverlässig sind. Die Daten werden so aufbereitet, dass sie präzise durchsucht und benutzerfreundlich dargestellt werden können.

Der Nutzen zeigt sich auf mehreren Ebenen. Für Studieninteressierte bedeutet der Chatbot eine erhebliche Erleichterung, da Informationen zu Studiengängen, Modulen und organisatorischen Abläufen ohne lange Suche zugänglich sind. Für Studierende bietet er direkten Zugriff auf Studien- und Prüfungsordnungen und spart wertvolle Zeit im Studienalltag. Auch die Hochschule selbst profitiert, weil die Verwaltung und Studienberatung durch die Automatisierung von Standardanfragen entlastet werden. Darüber hinaus stärkt der Einsatz eines modernen Chatbots das digitale und innovative Erscheinungsbild und steigert die Attraktivität für künftige Studierende.

Insgesamt verbindet FhatGPT technische Innovation mit praktischem Nutzen. Er erleichtert die Informationssuche, steigert die Effizienz in der Kommunikation und bietet einen hohen Mehrwert für alle Beteiligten. Für uns als Team ist die Entwicklung dieses Chatbots eine wertvolle Gelegenheit, unsere Kenntnisse in künstlicher Intelligenz, Datenaufbereitung und Softwareentwicklung zu vertiefen. Gleichzeitig entsteht ein Produkt, das die Servicequalität der Fachhochschule Wedel nachhaltig verbessern kann.

1.1. Ablaufbedingungen

Für die Ausführung des Programms müssen folgende Bedingungen erfüllt sein:

- Programmiersprache: Python $\geq$ 3.9 (empfohlen: 3.10 oder 3.11)
- Betriebssystem: Das Programm ist betriebssystemunabhängig und läuft auf jedem System, solange die passende Python-Version installiert ist.
- Zusätzliche Anforderungen:
 - pip zur Installation von Abhängigkeiten
 - Internetzugang für die Nutzung der OpenAI-Schnittstelle

1.2. Programminstallation/Programmstart

1. Virtuelle Umgebung erstellen im Projektordner um darauf die benötigten Abhängigkeiten zu installieren: `python -m venv venv`
2. Virtuelle Umgebung aktivieren mit Windows: `venv\Scripts\activate` ; unter Linux: `source venv/bin/activate`
3. Benötigte Abhängigkeiten installieren, die in der requirements.txt vorgegeben sind: `pip install -r requirements.txt`
4. OpenAI API-Key generieren auf <https://platform.openai.com/api-keys>
5. OpenAI API-Key hinterlegen in .streamlit/secrets.toml: `API_KEY = "dein-openai-api-key"`
6. Programm starten: `python app.py`
7. Programm richtet sich nun automatisch selbst ein und ist daraufhin nutzbar

1.3. Bedienungsanleitung

Die folgende Bedienungsanleitung erklärt die Nutzung des FH Wedel Chatbots. Sie enthält alle Informationen, die notwendig sind, um den Chatbot sicher und effizient einzusetzen. Ziel ist es, den Benutzern eine klare Orientierung zu geben, damit sie ihre Fragen rund um das Studium an der Fachhochschule Wedel schnell und zuverlässig beantwortet bekommen.

1.3.1. Zielsetzung des Chatbots

Der FH Wedel Chatbot wurde entwickelt, um Studierenden und Interessierten einen direkten Zugang zu wichtigen Informationen zu ermöglichen. Er beantwortet Fragen auf Grundlage offizieller Unterlagen der Hochschule, wie zum Beispiel Modulhandbücher oder Studienordnungen. Damit entfällt die mühsame Suche in umfangreichen Dokumenten. Der Chatbot soll eine Unterstützung bieten, ersetzt jedoch keine individuelle Studienberatung.

1.3.2. Grundlegende Regeln

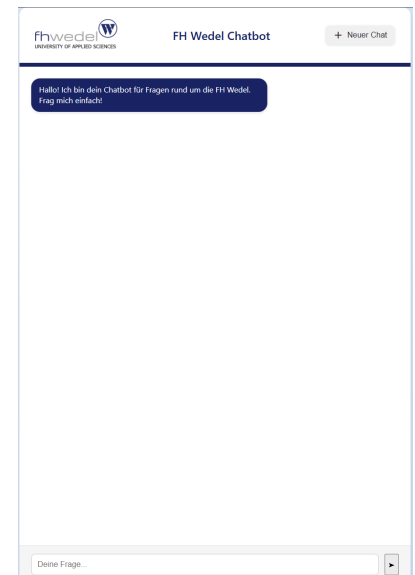
Der Chatbot liefert ausschließlich Antworten, die in den hinterlegten Dokumenten enthalten sind. Sind dort keine passenden Informationen zu finden, weist er darauf hin. Es werden keine Informationen erfunden oder aus dem Internet ergänzt. Damit die Antworten präzise ausfallen, empfiehlt es sich, Fragen möglichst konkret zu stellen.

1.3.3. Aufbau der Benutzeroberfläche

Die Benutzeroberfläche ist klar und übersichtlich gestaltet. Im oberen Bereich der Seite befindet sich eine Kopfzeile mit dem Logo der Fachhochschule auf der linken Seite, der Bezeichnung „FH Wedel Chatbot“ in der Mitte sowie ein Button für den Start eines neuen Chats auf der rechten Seite.

Darunter folgt der zentrale Chatbereich. Hier erscheinen die Fragen der Nutzer und die Antworten des Chatbots in Form von Sprechblasen. Die Gestaltung macht deutlich sichtbar, wer welche Nachricht geschrieben hat.

Am unteren Rand der Seite befindet sich das Eingabefeld. Dort können Fragen direkt eingetippt und über die Enter-Taste oder durch Klicken auf den Absende-Button mit dem Pfeilsymbol gesendet werden.



1.3.4. Nutzung des Chatbots

Nach dem Aufrufen der Startseite begrüßt der Chatbot die Benutzer mit einer kurzen Willkommensnachricht. Anschließend können Fragen gestellt werden. Während der Verarbeitung wird eine kleine Animation angezeigt, die signalisiert, dass die Antwort vorbereitet wird. Danach erscheint die Antwort schrittweise im Chatfenster, sodass der Text bereits lesbar ist, bevor er vollständig übertragen wurde.

Wenn eine Unterhaltung beendet oder von vorne begonnen werden soll, kann der Button „Neuer Chat“ in der Kopfzeile genutzt werden. Der bisherige Verlauf wird dann gelöscht und eine neue Unterhaltung beginnt mit der Standardbegrüßung.

1.4. Fehlermeldungen

Im Verlauf der Nutzung des FH Wedel Chatbots können verschiedene Fehlermeldungen auftreten. Die folgende Tabelle listet diese Meldungen auf, erläutert die möglichen Ursachen und zeigt, wie das Problem behoben werden kann.

Fehlermeldung	Ursache	Lösung
! Bitte gib eine Frage ein.	Das Eingabefeld wurde leer abgeschickt, ohne dass eine Frage formuliert wurde.	Eine vollständige Frage in das Eingabefeld eingeben und erneut absenden.
✗ Es ist ein Fehler aufgetreten. Bitte versuche es erneut.	Es besteht ein Verbindungsproblem zwischen der Benutzeroberfläche und dem Server. Dies kann durch eine instabile Internetverbindung oder eine kurzfristige Nichtverfügbarkeit der Anwendung verursacht sein.	Zuerst die Internetverbindung prüfen. Falls das Problem bestehen bleibt, über den Button „Neuer Chat“ eine neue Unterhaltung starten und die Anfrage erneut stellen.
! Entschuldigung, ich konnte keine Antwort generieren.	Der Chatbot hat die Anfrage zwar empfangen, konnte aber keine passende Antwort erzeugen. Dies geschieht oft bei unklaren oder mehrdeutigen Fragestellungen.	Die Frage präziser formulieren, zum Beispiel mit genaueren Angaben zum Thema oder zum Studiengang.
(Keine Antwort erscheint, Chat bleibt hängen)	Der Chatbot verarbeitet die Anfrage nicht oder die Verbindung ist unterbrochen.	Die Seite neu laden oder den Button „Neuer Chat“ verwenden, um den Gesprächsverlauf zurückzusetzen. Danach die Frage erneut eingeben.

2. Programmiererhandbuch

Dieses Programmiererhandbuch dokumentiert die Planung, Umsetzung und technische Grundlagen des entwickelten Chatbot-Projekts. Ziel ist es, künftigen Entwicklern – insbesondere Kommilitonen im Rahmen des Moduls *Software-Projekt* – eine verständliche und strukturierte Grundlage zur Einarbeitung in den Quellcode und die Architektur zu bieten.

Das Handbuch erläutert die eingesetzten Technologien, die Entwicklungsumgebung sowie die zentralen Entwurfsentscheidungen. Dadurch soll sichergestellt werden, dass die Weiterentwicklung, Pflege oder auch eine vollständige Neuerstellung der Anwendung nach einer längeren Pause mit minimalem Einarbeitungsaufwand möglich ist.

Bei der Erstellung wurde insbesondere darauf geachtet, nicht nur den aktuellen Stand, sondern auch die zugrunde liegenden Gedankengänge zu dokumentieren, sodass das Handbuch als Leitfaden und Nachschlagewerk gleichermaßen genutzt werden kann.

2.1. Entwicklungskonfiguration

Die nachfolgende Tabelle listet die exakten Softwarekomponenten, Frameworks und Bibliotheken auf, die für die Entwicklung des Projekts genutzt wurden. Diese Aufstellung ist notwendig, um eine reproduzierbare Entwicklungsumgebung zu gewährleisten.

Komponente	Version / Edition	Anmerkung
Betriebssystem	Windows 11 (64-Bit)	Hauptentwicklungsumgebung
IDE	IntelliJ IDEA Ultimate	Nutzung für Python- und Webentwicklung
Programmiersprache	Python 3.12	Grundlage für Backend und Flask-Server
Paketverwaltung	pip (in Python integriert)	Installation der Abhängigkeiten
Framework	Flask 3.1.2	Webserver für Chatbot-Oberfläche und API
OpenAI SDK	openai 1.102.0	Schnittstelle zur Nutzung von OpenAI-Assistenten
Scheduling-Library	schedule 1.2.2	Für geplante Synchronisationen
Konfigurationsdateien	toml 0.10.2	Für Secrets- und Settings-Verwaltung
Versionsverwaltung	Git	Projektverwaltung und Versionskontrolle

2.2. Problemanalyse und Realisation

Im Laufe der Entwicklung des Chatbots wurde mit verschiedenen Sprachmodellen und Frameworks experimentiert. Dabei ergaben sich unterschiedliche Problemstellungen, die im Folgenden analysiert werden. Diese Erfahrungen waren wichtig, um schrittweise eine robuste und performante Lösung zu entwickeln, die den Anforderungen der FH Wedel gerecht wird.

2.2.1. Nutzung von Gemini mit Streamlit

Die erste lauffähige Version des Chatbots wurde mit dem Sprachmodell Gemini in Kombination mit dem Framework Streamlit umgesetzt. Streamlit bot den großen Vorteil, dass sehr schnell ein funktionierendes User Interface entwickelt werden konnte, ohne dass viel Aufwand in Frontend-Technologien investiert werden musste. Mit wenigen Codezeilen war es möglich, eine Eingabemaske für Fragen und ein Ausgabefenster für die Antworten zu erstellen. Damit ließ sich zügig ein Prototyp aufbauen, der das Grundprinzip des Chatbots demonstrierte und erste Tests mit realen Dokumenten ermöglichte.

Wichtig ist dabei zu erwähnen, dass in dieser Version bereits eine Vektordatenbank integriert wurde. Die Dokumente wurden vorverarbeitet, in kleinere Textabschnitte (Chunks) zerlegt und als Vektoren gespeichert. Auf diese Weise musste Gemini nicht bei jeder neuen Anfrage alle PDFs erneut einlesen, sondern konnte gezielt in den Vektoren nach relevanten Textstellen suchen. Das führte zu spürbar besseren Antwortzeiten, die für die ersten Testläufe ausreichend schnell waren.

Allerdings zeigte sich bei längerer Nutzung ein zentrales Problem: die Oberfläche war stark von Streamlit abhängig. Die von Streamlit vorgegebenen Layout- und Designmöglichkeiten waren sehr beschränkt. Für ein Projekt, das speziell auf die FH Wedel zugeschnitten werden sollte, war dies ein Hindernis. Eine ansprechende und wiedererkennbare Benutzeroberfläche, die sich am Corporate Design der Hochschule orientiert, ließ sich mit Streamlit nur sehr eingeschränkt umsetzen. Aus diesem Grund entstand der Entschluss, Streamlit durch ein eigenes Frontend zu ersetzen.

2.2.2. Eigenes Frontend mit Gemini

Die zweite Entwicklungsphase bestand darin, die Abhängigkeit von Streamlit aufzugeben und ein eigenes Frontend zu entwickeln. Dies bot deutlich mehr Freiheit in Bezug auf Gestaltung und Funktionalität. Mit einem maßgeschneiderten Frontend konnte die Anwendung besser an die Bedürfnisse der Hochschule angepasst werden, beispielsweise durch eine moderne Benutzerführung, die Einbindung eigener Design-Elemente und die bessere Kontrolle über die Darstellung des Chatverlaufs.

Mit diesem Schritt ging jedoch ein neues Problem einher. Obwohl auch in dieser Version die Vektordatenbank genutzt wurde, war die Integration von Gemini in das neue System wesentlich langsamer als in der Streamlit-Umgebung. Die Verarbeitung schien ineffizienter zu laufen: bei jeder Anfrage wirkte es so, als ob Gemini erneut große Teile der Dokumente durchsuchte, anstatt die Vektoren direkt zu nutzen. Die Folge waren deutlich längere Antwortzeiten, die im Durchschnitt bei bis zu 30 Sekunden pro Anfrage lagen. Für eine produktive Nutzung war dies nicht akzeptabel, da eine zu lange Wartezeit die Benutzererfahrung erheblich beeinträchtigt.

Damit zeigte sich, dass die reine Umstellung auf ein eigenes Frontend nicht ausreichend war. Es musste eine Lösung gefunden werden, wie die Dokumentensuche innerhalb der Vektordatenbank effizienter gestaltet werden konnte, um die Ladezeiten wieder zu reduzieren.

2.2.3. Einführung einer optimierten Vektordatenbank

Die dritte Phase konzentrierte sich daher auf die Optimierung des Dokumentenspeichers. Das Ziel war, die durch das eigene Frontend verursachten Performance-Verluste zu kompensieren und die Antwortzeiten auf ein akzeptables Niveau zurückzuführen.

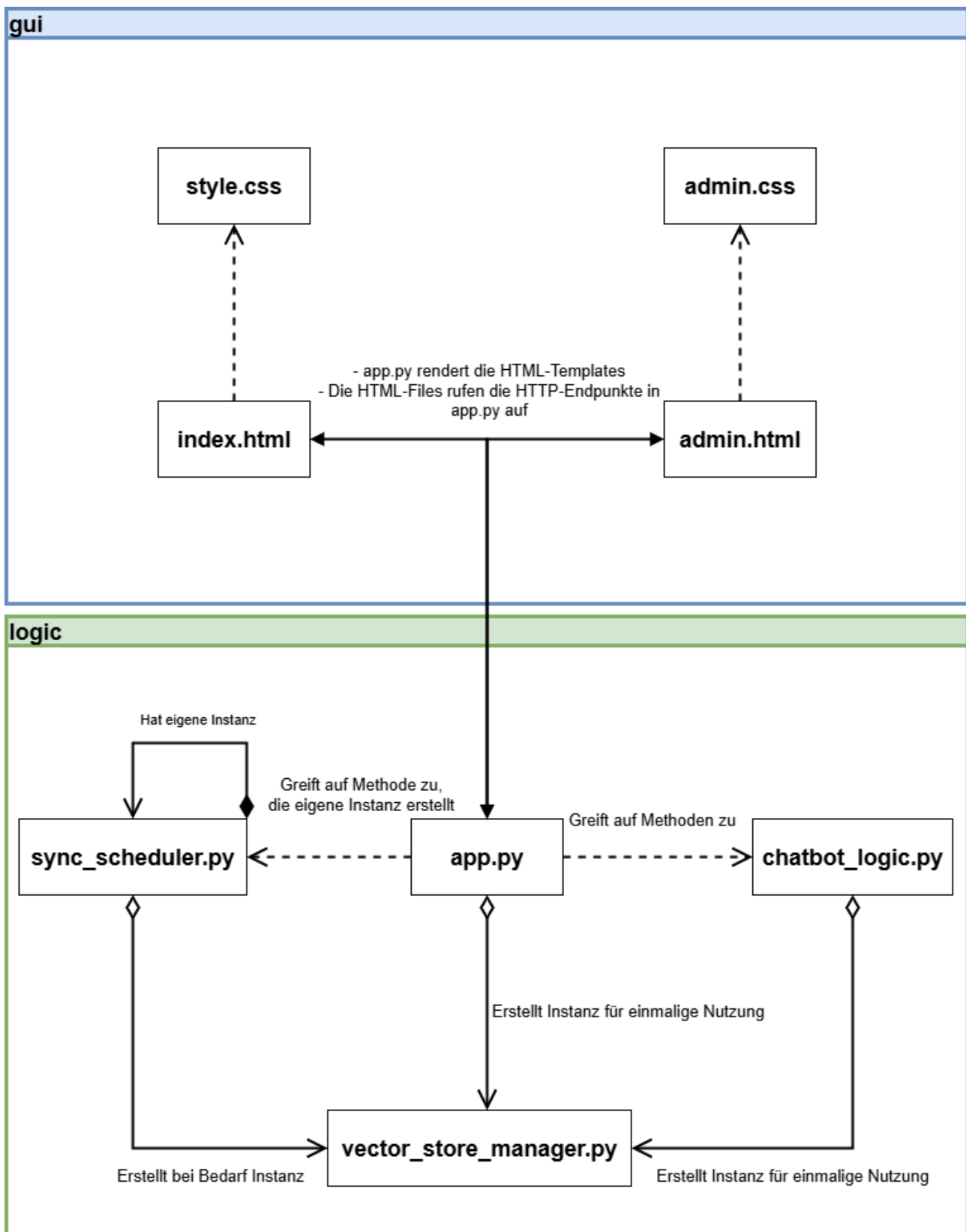
Die gewählte Lösung bestand darin, die bereits vorhandene Vektordatenbank weiter auszubauen und die Chunks noch strukturierter zu verwalten. Anstatt die gesamten Dokumente bei jeder Anfrage neu in den Prozess einzubringen, wurden die Daten lokal in Vektoren gespeichert und wiederverwendet. Durch diese Vorverarbeitung konnte Gemini gezielt nur die relevanten Abschnitte durchsuchen, anstatt jedes Mal alle PDFs einzubeziehen.

Der Effekt war deutlich messbar: die durchschnittliche Antwortzeit sank von etwa 30 Sekunden auf rund 12 Sekunden pro Anfrage. Zwar war dies noch nicht so schnell wie in der ursprünglichen Streamlit-Version, in der die Integration reibungsloser funktioniert hatte, aber die Verbesserung stellte einen wichtigen Schritt in Richtung eines praxistauglichen Systems dar. Zudem eröffnete die optimierte Vektordatenbank langfristig die Möglichkeit, weitere Sprachmodelle effizient einzubinden, ohne jedes Mal wieder an die Grenzen der Performance zu stoßen.

2.2.4. Nutzung von DeepSeek

Die Untersuchung der DeepSeek API hat gezeigt, dass die Kosten pro Token deutlich günstiger sind als bei Gemini und ChatGPT und die Qualität der generierten Ergebnisse insgesamt sehr gut war. Allerdings liegt die entscheidende Einschränkung im verfügbaren Kontextfenster: Mit rund 64.000 Tokens erlaubt DeepSeek deutlich weniger Eingaben als ChatGPT (ca. 128.000 Tokens) oder Gemini (über 1.000.000 Tokens). Für unseren Anwendungsfall ist ein großes Kontextfenster jedoch unerlässlich, da wir in einer Anfrage viele Informationen aus der Vektordatenbank übergeben müssen, um präzise und hochwertige Antworten zu erhalten. Dadurch erweist sich DeepSeek trotz seiner Kostenvorteile und guten Ergebnisse letztlich als ungeeignet für unseren Einsatzzweck.

2.3. Programmorganisationsplan



2.3.1 app.py

Das Hauptmodul fungiert als zentraler Einstiegspunkt der gesamten Anwendung und initialisiert damit auch die wichtigsten Klassen und Elemente. Es initialisiert die Flask-Webanwendung und stellt HTTP-Endpunkte für den Eingaben des Benutzers bereit. Zu den Kernaufgaben gehören:

- Initialisierung und Verwaltung des OpenAI-Clients
- Generierung und Lesen des Admin-Authentifizierungstokens
- Bereitstellung der HTTP-Endpunkte für Chat-Interaktionen (/ask, /reset)
- Bereitstellung der HTTP-Admin-Endpunkte für Systemverwaltung (/admin/status, /admin/sync, /admin/reset)
- Session-Management für Chat-Verläufe
- Integration des automatischen Synchronisations-Schedulers

2.3.2 chatbot_logic.py

Dieses Modul kapselt die gesamte KI-Interaktionslogik und verwaltet die Kommunikation mit der OpenAI Assistant API. Die Hauptfunktionalitäten umfassen:

- Erstellung und Konfiguration des OpenAI-Assistenten mit spezifischen Instruktionen für die FH Wedel
- Implementierung des Streaming-Mechanismus für Echtzeitantworten
- Automatische Verknüpfung des Assistenten mit dem aktuellen Vector Store

2.3.3 vector_store_manager.py

Das Modul implementiert einen Synchronisationsmechanismus zwischen lokalen PDF-Dokumenten und dem OpenAI Vector Store:

- Hash-basierte Änderungserkennung um Änderungen an Dokumenten zu erkennen
- Manifest-Verwaltung zur Verfolgung von Dateizuständen und Änderungen
- Upload und Löschen aus dem OpenAI Vector Store
- Detaillierte Statusrückgabe des Vector Stores für Admin-Oberfläche
- Reset-Funktion

2.3.4 sync_scheduler.py

Implementiert die automatisierte Hintergrundsynchrisation, um zu gewährleisten, dass lokale Dateien und Vektore Store synchron bleiben. Das Modul bietet:

- Konfigurierbares Intervall-basiertes Scheduling (Standard: 30 Minuten)
- Synchronisation bei Dateiänderungen
- Thread-basierte Ausführung ohne Blockierung der Hauptanwendung

2.4. Dateien

- requirements.txt:
 - Enthält die benötigten Python-Abhängigkeiten mit Versionsangaben.
 - Wird verwendet, um die Entwicklungsumgebung mit `pip install -r requirements.txt` aufzusetzen.
- README.md:
 - Projektbeschreibung, Features und Installationshinweise.
 - Übersicht der Projektstruktur und kurze Erklärung der enthaltenen Dateien und Ordner.

- `.gitignore`:
 - Definiert Dateien und Ordner, die nicht ins Repository eingchecked werden sollen.
 - Typische Inhalte: Editor-/IDE-Ordner, Cache-Dateien, virtuelle Umgebungen, temporäre Skripte.
- `/pdfs`:
 - Enthält die lokalen PDF-Dokumente, die vom Chatbot genutzt werden.
 - Strukturierte Unterordner für verschiedene Dokumenttypen (z. B. Modulhandbuch, Studienordnung, Regularien).
- `secrets.toml`:
 - Konfigurationsdatei für geheime Zugangsdaten (z. B. API-Keys).
 - Wird lokal angelegt und nicht ins Repository eingchecked.
- `Tests.xlsx`:
 - Enthält die durchgeführten Tests
- `app.wsgi`:
 - Stellt die Flask-Anwendung für den Server bereit und bindet die virtuelle Python-Umgebung ein.
 - Wird vom Apache-Modul `mod_wsgi` verwendet, um die Anwendung auszuführen.
- `fhatspt.conf`:
 - auf dem Server im Ordner `“/etc/apache2/sites-available”`
 - Apache-Konfigurationsdatei für die Bereitstellung der Anwendung über einen virtuellen Host.
 - Legt die WSGI-Anbindung fest, regelt den Zugriff auf statische Dateien und definiert Pfade für Logdateien.

2.5. Programmtests

Im Verlauf des Projekts wurden verschiedene Sprachmodelle hinsichtlich ihrer Eignung untersucht. Dabei rückten *Gemini 1.5 Flash* und *GPT-4o* in die engere Auswahl und wurden im Anschluss ausführlich getestet. Diese Tests sind in der Datei *Tests.xlsx* hinterlegt.

Ziel der Tests war es, die Richtigkeit beider Modelle bei der Beantwortung von Fragen zu den hinterlegten PDF-Dokumenten zu untersuchen. Informationen, die zwar inhaltlich richtig sind, aber nicht aus den zugrunde liegenden PDF-Dokumenten entnommen, sondern vom Modell ausgedacht und ergänzt wurden, wurden als falsch gewertet. Es wurden keine Folgefragen gestellt, sondern nur die erste Antwort gewertet. Falls ein Modell eine Klarstellung verlangt hat, wurde die Antwort als falsch bewertet, um Antworten verschiedener Modelle vergleichbar zu behalten.

Es wurden insgesamt 96 Fragen aus den folgenden Bereichen getestet:

- Modulhandbuch
- Studienordnungen
- Studienverlaufsplan
- Prüfungsverfahrensordnung
- Regularien
- Technisches Grundpraktikum
- Zulassungsordnung
- Die Auswertung ergab folgende Ergebnisse:
- Gemini 1.5 Flash hat 27,08% korrekt beantwortet
- GPT-4o hat 81,25% korrekt beantwortet

2.6 Server

Der Server des Projekts läuft auf einer virtuellen Maschine und ist unter `fhavgpt.studproj.fh-wedel.de` vor Ort aus der Fachhochschule Wedel oder per VPN erreichbar.

Die Projektdateien werden im Home-Verzeichnis unter dem Ordner `/home/projekt/FHATGPT/` entwickelt und verwaltet. Für die Ausführung auf dem Apache-Webserver müssen die aktualisierten Dateien in das Server-Verzeichnis `/var/www/FHATGPT/` übertragen werden. Aufgrund der Konfigurationsdateien müssen gegebenenfalls absolute Pfade in den Projektdateien verwendet werden.

- Git-Verzeichnis: `/home/projekt/FHATGPT/`
 - Hier befindet sich die lokale Kopie des Git-Repositories. Änderungen und Aktualisierungen erfolgen in diesem Verzeichnis.
- Server-Verzeichnis: `/var/www/FHATGPT/`
 - Dieses Verzeichnis wird vom Apache-Server verwendet, um die Anwendung über `mod_wsgi` bereitzustellen. Die Datei `app.wsgi` und alle notwendigen Projektdateien müssen hier verfügbar sein.
- Apache verwalten
 - Der Apache-Webserver kann über `systemctl` gesteuert werden:
 - `systemctl status apache2` zeigt den aktuellen Status des Servers
 - `systemctl start apache2` startet den Server
 - `systemctl stop apache2` stoppt den Server